

实验六 Block Nested Loop Join 环境搭建说明

本文档说明实验六所需的 PostgreSQL 12.0 本地开发环境搭建流程。实验要求在 PostgreSQL 查询执行器中实现 Block Nested Loop Join，并通过不同 block size 的运行结果比较性能差异。因此，环境搭建的目标不是安装一个普通数据库服务，而是准备一个可以编译、调试、反复安装并运行自定义 PostgreSQL 源码的实验环境。

实验步骤主要参考 `实验六.pdf` 中的 Preparation、Building Postgres、Starting PostgreSQL Server、Loading Test Data 与 Experiments 部分。

1. 实验背景

PostgreSQL 的 simple nested loop join 会对外表中的每一行重新扫描一次内表。若外表行数较多，该方法会产生大量重复扫描和磁盘 I/O。Block nested loop join 的思想是一次缓存一组外表元组，再用这一组元组与内表进行匹配，从而减少内表扫描次数。

本实验需要修改 PostgreSQL 12.0 的查询执行器实现，使 nested loop join 支持按块处理外表元组。不同 block size 不应改变连接结果，只会影响执行顺序和性能表现。实验最终需要记录 block size 为 `1`、`2`、`8`、`64`、`128`、`1024` 时的运行时间和 shared blocks read，并解释性能差异。

2. 准备材料

当前目录应包含以下文件或目录：

路径	作用
<code>postgresql-12.0/</code>	PostgreSQL 12.0 源码目录，后续编译与源码修改均在此目录内进行
<code>similarity_data.sql</code>	实验测试数据，用于创建并填充测试表
<code>实验六.pdf</code>	实验说明原文

建议在开始编译前确认目录：

```
1 | ls
```

3. 安装系统依赖

PostgreSQL 源码编译至少需要 C 编译器、`make`、`readline` 和 `zlib`。本实验还需要调试源码，因此建议安装调试器。

windows 系统可以采用 Ubuntu LTS，可以在 Microsoft Store 中直接下载。

3.1 Ubuntu / Linux

```
1 | sudo apt-get update
2 | sudo apt-get install build-essential libreadline-dev zlib1g-dev gdb
```

说明：

依赖	用途
<code>build-essential</code>	提供 <code>gcc</code> 、 <code>make</code> 等基础编译工具
<code>libreadline-dev</code>	支持 <code>psql</code> 命令行历史、编辑和补全功能
<code>zlib1g-dev</code>	提供 PostgreSQL 构建所需的压缩库
<code>gdb</code>	用于跟踪 PostgreSQL 后端进程和调试 <code>executor</code> 代码

如果系统提示缺少 `bison` 或 `flex`，可额外安装：

```
1 | sudo apt-get install bison flex
```

本实验主要修改 `executor` 中的 C 代码，通常不需要重新生成语法或词法分析器。

3.2 macOS

实验文档以 Linux 环境为主。如果在 macOS 上完成实验，需要安装 Xcode Command Line Tools，并通过 Homebrew 安装 `readline` 和 `zlib`：

```
1 | xcode-select --install
2 | brew install readline zlib
```

Apple Silicon Mac 的 Homebrew 默认路径通常是 `/opt/homebrew`；Intel Mac 通常是 `/usr/local`。后续执行 `configure` 时需要显式指定这些库的位置。

4. 编译并安装 PostgreSQL

实验要求从源码编译 PostgreSQL，并安装到当前用户家目录下的 `pgsql` 目录。这样可以避免影响系统已有 PostgreSQL，也便于反复安装修改后的版本。

4.1 Linux 编译安装

```
1 | cd postgresql-12.0
2 | ./configure --enable-depend --enable-cassert --enable-debug CFLAGS="-O0" --prefix=$HOME/pgsql \
3 | make
4 | make install
```

4.2 macOS 编译安装

Apple Silicon Mac 可使用：

```
1 | cd postgresql-12.0
2 | ./configure --enable-depend --enable-cassert --enable-debug CFLAGS="-O0" --prefix=$HOME/pgsql \
3 |   --with-includes="/opt/homebrew/opt/readline/include /opt/homebrew/opt/zlib/include" \
4 |   --with-libraries="/opt/homebrew/opt/readline/lib /opt/homebrew/opt/zlib/lib"
5 | make
6 | make install
```

Intel Mac 可将命令中的 `/opt/homebrew` 替换为 `/usr/local`。

4.3 编译参数说明

参数	说明
<code>--prefix=\$HOME/pgsql</code>	将 PostgreSQL 安装到用户家目录，避免使用系统目录
<code>--enable-debug</code>	生成调试符号，便于使用 <code>gdb</code> 或 <code>lldb</code> 跟踪源码
<code>CFLAGS="-O0"</code>	关闭编译优化，使调试时的源码行号和变量状态更直观
<code>--enable-cassert</code>	启用 PostgreSQL 内部断言，便于尽早发现实现错误
<code>--enable-depend</code>	自动追踪头文件依赖，修改头文件后相关源文件可自动重新编译
<code>--with-includes</code> / <code>--with-libraries</code>	macOS 下用于指定 Homebrew 安装的依赖库路径

`make` 成功完成后，终端通常会输出类似 `All of PostgreSQL successfully made. Ready to install.` 的提示。之后执行 `make install`，把编译结果安装到 `$HOME/pgsql`。

5. 初始化并启动数据库

编译安装完成后，需要初始化一个本地数据库集群。`initdb` 只需在首次创建数据目录时执行一次；之后修改源码并重新安装 PostgreSQL 时，通常不需要重新初始化，也不需要重新导入数据。

```
1 $HOME/pgsql/bin/initdb -D $HOME/pgsql/data --locale=C
```

启动 PostgreSQL：

```
1 $HOME/pgsql/bin/pg_ctl -D $HOME/pgsql/data -l /tmp/pg_logfile start
```

连接并检查版本：

```
1 $HOME/pgsql/bin/psql postgres -c "SELECT version();"
psql (15.1)
Type "help" for help.

postgres=#
```

停止 PostgreSQL：

```
1 $HOME/pgsql/bin/pg_ctl -D $HOME/pgsql/data stop
```

由于 PostgreSQL 被安装在 `$HOME/pgsql`，本文档中的命令均使用完整路径。若希望简化后续命令，可将其加入 `PATH`：

```
1 export PATH="$HOME/pgsql/bin:$PATH"
```

6. 创建实验数据库并导入数据

确保 PostgreSQL 服务已经启动，然后创建实验数据库：

```
1 $HOME/pgsql/bin/psql postgres -c "CREATE DATABASE similarity;"
psql (15.1)
Type "help" for help.

postgres=#
```

导入测试数据：

```
1 $HOME/pgsql/bin/psql similarity -f similarity_data.sql
```

也可以进入 `psql` 后执行：

```
1 \c similarity
2 \i similarity_data.sql
```

`similarity_data.sql` 会创建实验查询所需的数据表。若在空数据库中导入时看到 `DROP TABLE` 相关的 `table does not exist` 提示，通常是脚本先尝试删除旧表导致的，不影响后续建表和导入。

7. 后续源码修改与重新安装

环境搭建完成后，实验主体是阅读并修改 PostgreSQL executor 相关代码。实验说明中建议重点阅读以下文件：

文件	作用
<code>src/backend/executor/nodeNestloop.c</code>	nested loop join 的核心执行逻辑
<code>src/include/nodes/execnodes.h</code>	executor 状态结构定义，这里定义了 <code>NestLoopState</code> 可以修改
<code>src/backend/utils/misc/guc.c</code>	PostgreSQL 运行参数配置入口，可用于增加 block size 等实验参数
<code>src/backend/executor/execTuples.c</code>	TupleTableSlot 的创建、复制与管理
<code>src/backend/executor/execMain.c</code>	executor 主流程
<code>src/backend/executor/execProcnode.c</code>	executor 节点初始化和执行分发
<code>src/backend/executor/execScan.c</code>	扫描节点相关辅助逻辑
<code>src/include/executor/nodeNestloop.h</code>	nested loop 节点接口

按照“最小可完成实验”路径来做，只需要改变 nested loop executor 的执行方式，并用 GUC 参数控制 block size（本实验只要求实现 inner join）。核心文件修改文件：`nodeNestloop.c`、`execnodes.h`、`guc.c`；其他文件主要是阅读辅助。

修改源码后，应重新编译并安装：

```
1 cd postgresql-12.0
2 make
3 make install
```

随后重启数据库服务，使新的二进制生效：

```
1 $HOME/pgsql/bin/pg_ctl -D $HOME/pgsql/data restart -l /tmp/pg_logfile
```

一般情况下，源码修改后的循环是：

```
1 修改源码 -> make -> make install -> restart -> 运行测试
```

只有在重新创建数据目录、切换安装路径或数据库损坏时，才需要重新执行 `initdb` 和数据导入。

8. 运行实验查询

实验需要测试 nested loop join，因此应关闭 hash join 和 merge join，让优化器选择 nested loop join：

```
1 SET enable_hashjoin = off;
2 SET enable_mergejoin = off;
```

不要关闭 `enable_nestloop`，否则测试计划可能不再使用 nested loop join。

连接实验数据库并开启计时：

```
1 | $HOME/pgsql/bin/psql similarity
```

```
1 | \timing
2 | SET enable_hashjoin = off;
3 | SET enable_mergejoin = off;
```

运行实验查询两次，并记录第二次的耗时：

```
1 | SELECT count(*)
2 | FROM restaurantaddress ra, restaurantphone rp
3 | WHERE ra.name = rp.name;
4 |
5 | SELECT count(*)
6 | FROM restaurantaddress ra, restaurantphone rp
7 | WHERE ra.name = rp.name;
```

实验要求分别测试 block size 为 1、2、8、64、128、1024 的性能。每个 block size 下都应记录第二次运行的查询时间和 shared blocks read。第二次结果更适合记录，是因为第一次运行可能包含冷启动、缓存预热等额外开销。

如需确认查询计划，可使用：

```
1 | EXPLAIN
2 | SELECT count(*)
3 | FROM restaurantaddress ra, restaurantphone rp
4 | WHERE ra.name = rp.name;
```

输出中应能看到 Nested Loop。

9. 参考资料

以下资料可作为理解实验背景、实现思路和代码位置的辅助参考。不同教程中的实现方式可能存在差异，实际修改时应以 PostgreSQL 12.0 源码结构和实验要求为准。

- [PostgreSQL 源码编译安装相关教程](#)
- [PKXX1943/Block-Nested-Loop-Join-pgsql](#)
- [FragranceShang/Implement-block-nested-loop-join-algorithm-in-Postgres](#)

10. 常见问题

问题	处理方式
configure 提示缺少 readline	Ubuntu 安装 libreadline-dev；macOS 安装 readline 并在 configure 中指定 Homebrew 路径
configure 提示缺少 zlib	Ubuntu 安装 zlib1g-dev；macOS 安装 zlib 并指定库路径
修改源码后行为没有变化	确认已经执行 make install，并重启 PostgreSQL 服务
查询计划不是 Nested Loop	确认关闭的是 enable_hashjoin 和 enable_mergejoin，不要关闭 enable_nestloop
导入数据时出现旧表不存在	若后续建表和导入成功，可忽略脚本开头删除旧表导致的提示

11. 流程概览

```
1  安装依赖
2      -> configure
3      -> make
4      -> make install
5      -> initdb
6      -> pg_ctl start
7      -> CREATE DATABASE similarity
8      -> 导入 similarity_data.sql
9      -> 修改 PostgreSQL executor 源码
10     -> make && make install
11     -> pg_ctl restart
12     -> 运行实验查询并记录性能
```